

Naive Bayes & Numerical Data

1 Why we need special handling

- Naive Bayes is built on **probability of features given a class**:

$$P(x_i | y)$$

- For **categorical** features, we can directly estimate probabilities by counting.
- For **continuous / numerical** features, counting each unique value is impossible → we need to **model the probability distribution** of each feature for each class.

2 Gaussian (Normal) Naive Bayes

- The most common approach for numeric data is to **assume each feature follows a Gaussian distribution** inside every class.

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_{y,i}^2}} \exp\left(-\frac{(x_i - \mu_{y,i})^2}{2\sigma_{y,i}^2}\right)$$

Where:

- $\mu_{y,i}$ = mean of feature i in class y
- $\sigma_{y,i}^2$ = variance of feature i in class y

The algorithm **learns μ and σ** for each (feature, class) pair from the training data.

3 Putting it together

- For each class:
 - Compute prior probability $P(y)$.
 - For each numeric feature, estimate μ and σ .
- To predict a new sample:
 - Plug the feature values into the Gaussian formula above for each class.
 - Multiply all feature likelihoods (or sum log-likelihoods).
 - Choose the class with the highest posterior probability.

4 When Gaussian isn't ideal

- If the feature distribution is **not bell-shaped**, other options:
 - Kernel Density Estimation (KDE)** Naive Bayes
 - Discretization** (bin numerical values into categories)
 - Transform data (e.g., log-scale) to make distribution more Gaussian-like.

5 scikit-learn implementation

- GaussianNB handles continuous data automatically:

```
from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

✓ Key Takeaways

- Naive Bayes handles numerical features by **modeling their distribution** commonly as **Gaussian**.
 - Each feature is assumed to be **independent given the class** (the “naive” assumption).
 - Estimating μ and σ from training data makes probability computation straightforward.
-